Online Appendix

Finding the Sweet Spot: Optimizing Batch Size for Accurate and Cost-Efficient LLM-Based Crowd Requirements Classification

Anonymous Author(s)

Author affiliations omitted for double-blind review

This appendix collects material referenced from the main paper but omitted there for space. Section A gives dataset examples; Section B the per-class recall analysis behind RQ2; Section C the per-prompt Macro F1 breakdown behind RQ1 and RQ4; Section D the full efficiency ranking behind RQ3; and Section E the failure taxonomy and error analysis. All numbering, terminology, and metric definitions follow the main paper.

A Dataset Examples

The experimental data are real-world, crowd-generated smart-home requirements drawn from a publicly available CrowdRE dataset of 2,966 requirements [1]. Each entry is a user story annotated with a role, a feature, a benefit, and an application domain. Following the prior study [2], the text to be classified merges the feature and benefit attributes into a single requirement, and the application domain is used as the sector label. Table 1 shows one example per sector.

Table 1. Examples from the CrowdRE dataset [1].

ID	Requirement	Sector
R1	my smart home to start laundry after it senses a full load of clothes, I can save time and manage my water use efficiently.	Energy
R2	my smart home to sync with my biorhythm app and turn on some music that might suit my mood when I arrive home from work, I can be relaxed.	Entertainment
R3	my medicine pill holder to notify me about the pill that are going to expire, So that I can replace the old one and purchase new one.	Health
R4	my smarthome to alert me whenever someone sets up a spycam inside my home, I won't be monitored.	Safety
R5	my fridge to tell me when I am running low on food, I can restock.	Other

B Per-Class Recall on the Quinary Task (RQ2)

RQ2 in the main paper reports that larger batches redistribute predictions from the specific sectors into the low-precision catch-all "Other" class. Table 2 gives the per-class recall underlying that claim, averaged over the five models, at a batch of one and a batch of sixty-four, broken down by prompt.

Table 2. Per-class recall on the Quinary task (five-model average) for batch sizes $B = 1$ and $B = 64$.

B	Prompt	Energy	Entertainment	Health	Safety	Other
1	EZS	0.74	0.69	0.67	0.89	0.56
1	FSR	0.81	0.64	0.54	0.82	0.73
1	Avg	0.78	0.66	0.60	0.85	0.64
64	EZS	0.71	0.59	0.54	0.82	0.70
64	FSR	0.73	0.57	0.52	0.84	0.71
64	Avg	0.72	0.58	0.53	0.83	0.71

Reading the average rows, every well-defined class loses recall as the batch grows, with Entertainment and Health falling the most (by 0.08 and 0.07 respectively), while the recall of the catch-all "Other" class rises from 0.64 to 0.71. The same shift appears in the raw prediction counts pooled over both prompts:[1] the number of items predicted as "Other" grows by 16% (from 940 to 1,090), the share of all misclassifications that fall into "Other" rises from 56% to 61%, and the precision of "Other" stays low throughout (0.44 at a batch of one and 0.42 at a batch of sixty-four). Because Macro F1 is the mean of the per-class scores, these recall losses are exactly what the aggregate drop reported in RQ2 reflects. This redistribution has no counterpart on the Quaternary task, which by design contains no catch-all class: there the misclassifications stay spread across the four specific sectors, with no single class taking more than about a third of them at either batch size, so no sink forms.

C Per-Prompt Macro F1 Breakdown (RQ1, RQ4)

Table 2 of the main paper reports Macro F1 averaged over the two prompting strategies. Tables 3 and 4 give the underlying per-prompt values for the Quaternary and Quinary tasks respectively, listing the Enhanced Zero-Shot (EZS) prompt, the Few-Shot with Reasoning (FSR) prompt, and their average for every model at every batch size. These are the values behind the RQ4 comparison, including the observation that at a batch of thirty-two on the Quinary task the EZS rows average 0.720 Macro F1 against 0.702 for the FSR rows.

D Cost, Latency, and the Full Efficiency Ranking (RQ3)

D.1 Per-Requirement Cost and Execution Time by Batch Size

Table 5 gives the aggregate cost and latency behind the headline efficiency figures quoted in the abstract and conclusion of the main paper. Each cell is averaged over both tasks, both prompts, and all five default models, so the values describe general behaviour across the model pool rather than any single configuration.

[1] `https://github.com/anonymousscot-jpg/llm-batch-size-crowdre/blob/main/results/Quinary\_confusion\_pooled.csv`

Table 3. Macro F1 on the Quaternary task: EZS, FSR, and their average per model. Bold marks the best batch on the average row.

Model	Prompt	1	2	4	8	16	32	64
Gemma-4-31B	EZS	.871	.840	.831	.834	.843	.865	.839
	FSR	.858	.840	.826	.836	.811	.820	.833
	Avg	**.864**	.840	.828	.835	.827	.842	.836
Mixtral-8x22B	EZS	.845	.804	.769	.785	.780	.712	.737
	FSR	.836	.787	.731	.740	.743	.722	.760
	Avg	**.840**	.795	.750	.762	.761	.717	.748
Nemotron-U.	EZS	.862	.843	.848	.818	.822	.816	.806
	FSR	.798	.802	.788	.830	.820	.833	.800
	Avg	**.830**	.823	.818	.824	.821	.824	.803
Llama-4-Mav.	EZS	.818	.827	.840	.840	.862	.862	.828
	FSR	.851	.841	.841	.849	.873	.862	.861
	Avg	.835	.834	.840	.844	**.868**	.862	.844
DeepSeek-V3.2	EZS	.812	.821	.830	.824	.819	.846	.840
	FSR	.835	.839	.843	.840	.823	.843	.827
	Avg	.824	.830	.836	.832	.821	**.844**	.834

Table 4. Macro F1 on the Quinary task: EZS, FSR, and their average per model. Bold marks the best batch on the average row.

Model	Prompt	1	2	4	8	16	32	64
Gemma-4-31B	EZS	.730	.740	.746	.743	.732	.744	.739
	FSR	.729	.705	.719	.723	.730	.712	.716
	Avg	.730	.722	.733	**.733**	.731	.728	.727
Mixtral-8x22B	EZS	.693	.725	.693	.697	.677	.652	.599
	FSR	.704	.668	.670	.666	.678	.614	.616
	Avg	**.698**	.697	.681	.681	.678	.633	.607
Nemotron-U.	EZS	.727	.724	.723	.740	.737	.722	.704
	FSR	.688	.698	.702	.742	.700	.717	.695
	Avg	.708	.711	.712	**.741**	.719	.720	.699
Llama-4-Mav.	EZS	.711	.713	.709	.734	.723	.746	.722
	FSR	.730	.731	.706	.724	.734	.734	.720
	Avg	.721	.722	.707	.729	.728	**.740**	.721
DeepSeek-V3.2	EZS	.711	.726	.713	.730	.751	.734	.727
	FSR	.733	.710	.729	.715	.722	.731	.701
	Avg	.722	.718	.721	.722	**.737**	.732	.714

Table 5. Per-requirement token cost and execution time by batch size, each averaged over both tasks, both prompts, and all five default models.

Batch	1	2	4	8	16	32	64
Tokens/req	594.5	345.5	230.0	162.9	124.2	99.1	82.8
Time (s)	3196	2235	1696	1197	921	759	642

Moving from a batch of one to a batch of thirty-two reduces the per-requirement token cost from 594.5 to 99.1 tokens and the execution time from 3,196 s to 759 s, reductions of 83% and 76% respectively. These are the values underlying the "roughly 80%" token figure and the "roughly 76%" time figure reported in the main paper; the token figure is quoted there from the fitted amortization curve of Equation 2, which gives $C(1) = 599$ and $C(32) = 105.9$ and hence 82%, agreeing with the raw average to within about one point.

Execution times were obtained from the per-run `batch_results.csv` logs in the replication package, one file per model and task. For each of the five models and two tasks, runs were filtered to batch sizes of one and thirty-two, grouped by batch size and prompt, and the `time_seconds` column averaged across repetitions. This yields twenty cells at each batch size (five models × two tasks × two prompts), whose mean gives the values above. Wall-clock time is measured end-to-end and includes generation, network round-trips, rate-limit delays, retries, and validation, so it is an implementation-dependent proxy for deployment latency rather than a measure of raw model speed.

D.2 Full Efficiency Ranking

Table 3 of the main paper lists the five highest-scoring configurations by the efficiency score S. Table 6 gives the full top-ten ranking. Macro F1, tokens per requirement, and time are each averaged over both tasks and both prompts; "Pareto" marks the non-dominated configurations. Ranks 6–10 are all dominated and are included here only for completeness.

Table 6. The ten highest-scoring configurations by the efficiency score S.

			Avg. (both tasks & prompts)				
Rank	Model	Batch	F1	Tok/req	Time (s)	S	Pareto
1	Llama-4-Maverick	32	.801	105.9	361	.962	Yes
2	Llama-4-Maverick	16	.798	126.0	432	.923	No
3	Gemma-4-31B	32	.785	69.9	195	.867	Yes
4	Gemma-4-31B	64	.782	60.5	160	.849	Yes
5	Llama-4-Maverick	64	.783	89.1	267	.834	No
6	Gemma-4-31B	8	.784	103.8	339	.831	No
7	Gemma-4-31B	16	.779	82.9	260	.807	No
8	Nemotron-U.	8	.783	117.7	452	.804	No
9	Llama-4-Maverick	8	.787	160.6	689	.798	No
10	Gemma-4-31B	4	.781	144.2	405	.779	No

E Failure Taxonomy and Error Analysis

E.1 A Taxonomy of Batch-Level Failures

A useful way to read the results is to separate two kinds of error, *semantic* and *structural*. The prior study established that these models classify CrowdRE requirements accurately at a batch of one [2], so the degradation at large batches is

not a wholesale loss of capability. In our logs the errors take two observable forms. *Semantic attrition* is the larger of the two: the model returns a complete, valid response but assigns items to the wrong category. It never raises a pipeline error, appears only as a lower Macro F1, and dominates on the ambiguous Quinary task, where the displaced items concentrate in the "Other" class (Section B). *Structural failure* is rarer but absolute: the response cannot be used at all. Only 34 of the roughly 12,700 batches in the evaluation failed structurally, of which 31 were truncated or otherwise unparseable JSON, 2 carried an identifier that did not match the input, and 1 returned the wrong number of items. The structural failure *rate* rises at the largest batches: averaged over both prompts, it stays at or below 0.5% through a batch of sixteen and then climbs to about 1% at batches of thirty-two and sixty-four; under the concise prompt alone it reaches 2% at a batch of sixty-four. By raw count, however, most failures occur at small batches, where there are far more batches: 28 of the 34 fall at a batch of four or smaller.

E.2 Error Types in the Model's Own Words

The recorded outputs make these two types of error concrete.

The first shows the semantic side. On the Quinary task, the Few-Shot prompt asks Llama-4-Maverick to classify the requirement *"my music player to work in conjunction with the lights, to set the mood for when I am entertaining,"* and its reasoning trace weighs the ambiguity explicitly:

> *"... The core functionality here relates to enhancing the entertainment experience by integrating media (music) with environmental settings (lighting). Although it involves lighting, which could be associated with Energy, the context provided ('to set the mood for when I am entertaining') leans more towards creating an ambiance for Entertainment."*

Here, the model resolves the item correctly, explicitly rejecting the Energy reading that the word "lights" invites. The aggregate result of RQ2 is what this looks like when the borderline call goes the other way: as the batch grows, items like this one are increasingly routed to the catch-all "Other" class. The output stays valid throughout, which is why semantic attrition shows up only as a lower Macro F1 and never as a pipeline error.

The other two examples are structural. The rarer form is a malformed identifier: at a batch of one, DeepSeek-V3.2 wrapped the identifier in brackets,

```
FINAL_JSON:
[{"id": "[REQ_070]", "category": "Other"}]
```

so the returned `[REQ_070]` no longer matched the input `REQ_070` and the batch was rejected. The dominant form is truncation. At a batch of sixty-four on the Quinary task, Mixtral-8x22B emitted correct items but produced JSON that did not parse, leaving the array unterminated; the response used only about 3,584 tokens, far below the 15,616-token budget, so this is a generation defect rather than a budget exhaustion:

```
[{"id": "REQ_257", "category": "Energy"},
 ...
 {"id": "REQ_262", "category": "Safe"},
 ...
 {"id": "REQ_299", "category": "Energy"},
 {"id": "REQ_300", "cate
```

Two problems appear at once: a malformed label ("Safe" instead of "Safety") and a truncated final token that leaves the array unclosed. Under the zero-tolerance policy all sixty-four items are scored as incorrect even though most had already been classified before the cut-off. This is why batching is attractive yet brittle at the ceiling: the semantic work is largely sound, but a single structural break discards the whole batch.

F Prompt Templates

The full Enhanced Zero-Shot (EZS) and Few-Shot with Reasoning (FSR) templates, for both the Quaternary and Quinary tasks, together with the decoding settings and the response-validation rules, are provided as a separate document in the replication package.[2]

References

1. Murukannaiah, P.K., Ajmeri, N., Singh, M.P.: Acquiring creative requirements from the crowd: understanding the influences of personality and creative potential in Crowd RE. In: IEEE 24th International Requirements Engineering Conference (RE), pp. 176–185. IEEE (2016)
2. Anonymous, A.: An empirical analysis of large language models for sector classification of crowd-based software requirements. In: Proceedings of the 41st ACM/SIGAPP Symposium on Applied Computing (SAC '26). ACM (2026)

[2] https://github.com/anonymousscot-jpg/llm-batch-size-crowdre/blob/main/docs/prompt_templates.md